

420-445

Programmation dans un environnement
transactionnel

Plan du cours

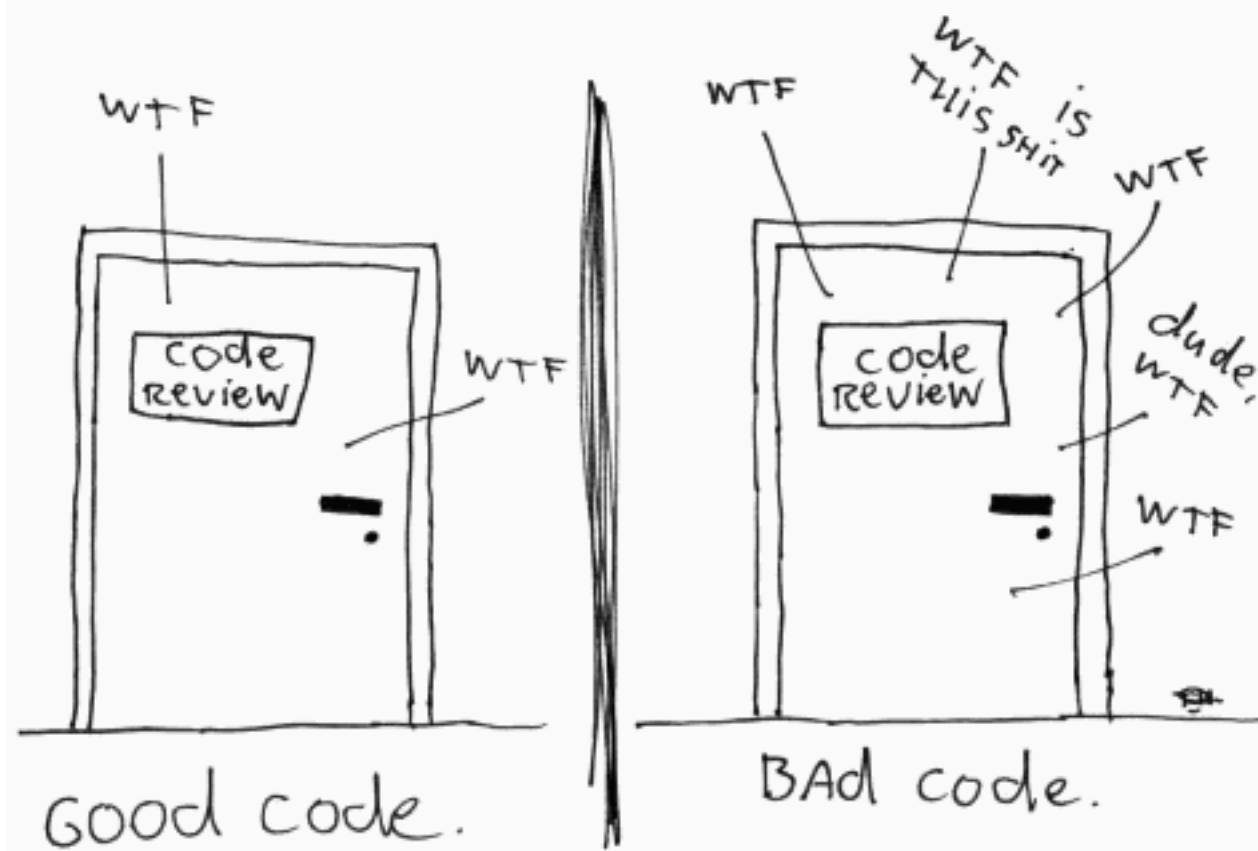
- Présentation du plan de cours
- Présentation du professeur et des élèves
- Discussion sur le développement d'applications transactionnelles
 - Développer seul vs en équipe + 'scope' des projets
 - Les tests

Plan du cours

- Discussion sur l'environnement de développement et les outils utilisés
 - Linux (ou WSL, ou au pire, GitBash)
 - Java17 (à l'aide de sdkman)
 - Git, GitHub
 - Maven (à l'aide de sdkman)
 - IntelliJ (Ultimate student edition)
 - Spring Boot (via maven)
 - React (via npm, npx...)
 - Node (<https://github.com/tj/n>)
 - Slack

Clean Code

The ONLY VALID MEASUREMENT
OF CODE QUALITY: WTFs/MINUTE



(c) 2008 Focus Shift

Clean Code

- Noms de variables significatifs
- Pas de MAGIC NUMBERS
- Méthodes très courtes (5 à 10 lignes)
- Un niveau de détail par méthode
- ‘Stair Step’ Les méthodes les plus importantes en haut suivies de leurs méthodes privées, celles-ci le plus proche possible des méthodes appelantes.
- Petites classes : SRP (Single Responsibility Principle)

Clean Code

- Extraire les conditions des 'if' et 'switch' en méthodes si elles sont trop compliquées.
- Remplacer les commentaires par des bons noms de méthodes.
- Formatage de type article de journal
- Éliminer le code mort
- Essayer de suivre CQRS (Command Query Responsibility Separation)
- 'Tell, don't ask'. Loi de demeter. (Train wreck)
- Remplacer si possible des 'if/else' avec des 'gard clauses'
- Éliminer les flags (boolean) des paramètres des méthodes
- Encapsulate conditionnals

Quel est le problème ?

```
class Calculator {  
    public static void main(String[] args) {  
        int a=5,b=10,c=15,d=20,e=25;  
        int sum=a+b+c+d+e;  
        System.out.println("The sum is: "+sum);  
    }  
}
```


Les noms doivent être significatifs

```
class Calculator {  
    public static void main(String[] args) {  
        int firstNumber = 5;  
        int secondNumber = 10;  
        int thirdNumber = 15;  
        int fourthNumber = 20;  
        int fifthNumber = 25;  
        int sum = firstNumber + secondNumber + thirdNumber + fourthNumber  
+ fifthNumber;  
        System.out.println("The sum is: " + sum);  
    }  
}
```


Quel est le problème ?

```
public class OrderProcessor {  
    public String processOrder(Order order, Inventory inventory,  
EmailClient emailClient) {  
        // Check if inventory has enough items  
        if (inventory.hasEnoughItems(order.getItems())) {  
            // Update inventory  
            inventory.update(order.getItems());  
            // Send email to customer  
            emailClient.sendOrderConfirmation(order.getCustomerEmail());  
            // Save order to database  
            order.save();  
            // Return success message  
            return "Order processed successfully!";  
        } else {  
            // Return error message  
            return "Error: Not enough items in inventory!";  
        }  
    }  
}
```

Quel est le problème ?

```
public class OrderProcessor {  
    public String processOrder(Order order, Inventory inventory,  
    EmailClient emailClient) {  
        // Check if inventory has enough items  
        if (inventory.hasEnoughItems(order.getItems())) {  
            // Update inventory  
            inventory.update(order.getItems());  
            // Send email to customer  
            emailClient.sendOrderConfirmation(order.getCustomerEmail());  
            // Save order to database  
            order.save();  
            // Return success message  
            return "Order processed successfully!";  
        } else {  
            // Return error message  
            return "Error: Not enough items in inventory!";  
        }  
    }  
}
```

- Commentaires inutiles. On devrait plutôt faire des ‘extract method’
- La méthode est trop longue
- La méthode fait trop de choses
- On pourrait avoir un ‘gard clause’ pour la vérification d’inventaire

Voici une proposition de réécriture

```
public void processOrder(Order order, Inventory inventory)
    throws InvalidCustomerException, OutOfStockException {
    if (!validateCustomer(order.getCustomer())) {
        throw new InvalidCustomerException();
    }

    if (!checkInventory(order.getItems())) {
        throw new OutOfStockException();
    }

    updateInventory(order.getItems(), inventory);
    saveOrder(order);
    sendConfirmationEmail(order.getCustomer().getEmail());
}
```

```
public class OrderProcessor {
    public boolean checkInventory(Order order, Inventory inventory) {
        return inventory.hasEnoughItems(order.getItems());
    }
    public void updateInventory(Order order, Inventory inventory) {
        inventory.update(order.getItems());
    }
    public void sendConfirmationEmail(Order order, EmailClient
emailClient) {
        emailClient.sendOrderConfirmation(order.getCustomerEmail());
    }
    public void saveOrder(Order order){
        order.save();
    }
}
```

Quel est le problème ?

```
class OrderProcessor {
    public String processOrder(Order order, Inventory inventory,
        EmailClient emailClient, Customer customer) {
        if (order.isValid()) {
            if (inventory.hasEnoughItems(order.getItems())) {
                if (customer.isPremium()) {
                    if (customer.hasEnoughCredit()) {
                        inventory.update(order.getItems());

                        emailClient.sendOrderConfirmation(order.getCustomerEmail());
                        order.save();
                        return "Order processed successfully!";
                    } else {
                        return "Error: Not enough credit!";
                    }
                } else {
                    if (customer.hasValidShippingAddress()) {
                        inventory.update(order.getItems());

                        emailClient.sendOrderConfirmation(order.getCustomerEmail());
                        order.save();
                        return "Order processed successfully!";
                    } else {
                        return "Error: Invalid shipping address!";
                    }
                }
            } else {
                return "Error: Not enough items in inventory!";
            }
        } else {
            return "Error: Invalid order!";
        }
    }
}
```

Remplacer les 'if' imbriqués par des 'returns'.

Ceci est également appelé 'gard clauses'

```
class OrderProcessor {
    public String processOrder(Order order, Inventory inventory,
        EmailClient emailClient, Customer customer) {
        if (!order.isValid()) {
            return "Error: Invalid order!";
        }
        if (!inventory.hasEnoughItems(order.getItems())) {
            return "Error: Not enough items in inventory!";
        }
        if (customer.isPremium() && !customer.hasEnoughCredit()) {
            return "Error: Not enough credit!";
        }
        if (!customer.isPremium() && !customer.isValidShippingAddress())
        {
            return "Error: Invalid shipping address!";
        }
        inventory.update(order.getItems());
        emailClient.sendOrderConfirmation(order.getCustomerEmail());
        order.save();
        return "Order processed successfully!";
    }
}
```

Quel est le problème ?

```
class Circle {  
    private double radius;  
    public Circle(double r) {  
        radius = r;  
    }  
    public double getArea() {  
        return 3.14159 * radius * radius;  
    }  
}
```

Remplacer les MAGIC NUMBERS par des constantes

```
class Circle {  
    private double radius;  
    private static final double PI = 3.14159;  
    public Circle(double r) {  
        radius = r;  
    }  
    public double getArea() {  
        return PI * radius * radius;  
    }  
}
```


Quel est le problème ?

```
class Bank {  
    private List<Account> accounts;  
  
    public void transferMoney(Account from, Account to, double amount) {  
        from.getBalance().withdraw(amount);  
        to.getBalance().deposit(amount);  
    }  
}  
  
class Account {  
    private Balance balance;  
  
    public Balance getBalance() {  
        return balance;  
    }  
}  
  
class Balance {  
    private double amount;  
  
    public void withdraw(double amount) {  
        this.amount -= amount;  
    }  
  
    public void deposit(double amount) {  
        this.amount += amount;  
    }  
}
```


Quel est la solution ?

```
} public void transferMoney(Account from, Account to, double amount) {  
    | from.withdraw(amount);  
    | to.deposit(amount);  
}  
}
```

3 usages

```
class Account {  
    | 3 usages  
    | private Balance balance;  
  
    | no usages  
    | public Balance getBalance() {  
    | | return balance;  
    | }  
}
```

1 usage

```
} public void withdraw(double amount) {  
    | balance.withdraw(amount);  
}
```

1 usage

```
} public void deposit(double amount) {  
    | balance.deposit(amount);  
}  
}
```

Clean Code

- <https://github.com/leonardolemie/clean-code-java#variables>
- <https://github.com/abiodunjames/Awesome-Clean-Code-Resources#java>